

访谈笔记

[Berkeley LLM Agents F24] Agents for Software Development —Graham Neubig

基于公开课程资料整理

2024 年 10 月 14 日



视频作者/频道: Berkeley RDI

发布日期: 2024 年 10 月 14 日

视频时长: 1:01:21

视频链接: <https://www.youtube.com/live/f9L9Fkq-8K4>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 引言：为什么软件开发 Agent 正在崛起	4
1.1 行业背景与平台	4
1.2 赋能普罗大众	4
1.3 本章小结	4
2 OpenHands 平台与 Agent 架构	4
2.1 OpenHands 架构组件	4
2.2 多模态接口与审计	5
2.3 本章小结	5
3 SWE-bench：软件工程评测的基准	6
3.1 Lite 与 Full 的演进	6
3.2 评估流程与指标	6
3.3 本章小结	7
4 Agent 工程实践：规划、执行与验证	7
4.1 Plan-Act-Verify 细节	7
4.2 人机协作与工具覆盖	8
4.3 本章小结	8
5 未来方向与治理挑战	9
5.1 Agent 训练与数据	9
5.2 治理与安全	9
5.3 本章小结	9
6 工程案例：Agent 参与的真实仓库	9
6.1 半自动化提交	9
6.2 过程回顾与反馈	9
6.3 本章小结	10
7 可观察性与调试实践	10
7.1 日志与监控流水线	10
7.2 调试模式与快速回滚	11
7.3 本章小结	11
8 代理指标与质量门控	11
8.1 关键指标	11
8.2 质量门控节点	11
8.3 本章小结	12
9 术语表与工具清单	12
9.1 常见术语	12
9.2 工具清单	12
9.3 本章小结	12

10 工程协作与变更流程	13
10.1 项目管理视角	13
10.2 交付与风险沟通	13
10.3 本章小结	13
11 访谈内容速览	13
11.1 0:00–15:00: 背景与平台	13
11.2 本章小结	14
12 部署与监控实践	14
12.1 CI/CD 集成	14
12.2 小规模实验	14
12.3 本章小结	14
13 访谈建议与常见问题	15
13.1 观众提问回顾	15
13.2 建议清单	15
13.3 本章小结	15
14 数据洞察与评估仪表板	15
14.1 核心仪表板视图	15
14.2 仪表板反馈循环	16
14.3 本章小结	16
15 团队文化与培训	16
15.1 文化落地策略	16
15.2 培训与指南	16
15.3 本章小结	17
16 安全治理与审查流程	17
16.1 审计链设计	17
16.2 审批与权限	17
16.3 本章小结	17
17 实战回顾与案例模板	17
17.1 代码修复示例	17
17.2 失败与纠错记录	18
17.3 本章小结	18
18 事件响应与复盘	18
18.1 故障归因流程	18
18.2 复盘纪要传播	18
18.3 本章小结	19

19 实施检查与持续改进	19
19.1 常规检查表	19
19.2 持续改进循环	19
19.3 本章小结	20
20 总结与延伸	20
20.1 本章小结	20

1 引言：为什么软件开发 Agent 正在崛起

1.1 行业背景与平台

Graham Neubig 先以 CMU 教授和 All Hands AI 的首席科学家的身份开场，讲述他在软件开发与人工智能之间的双重积累。All Hands AI 正在维护开源的 OpenHands（前身 OpenDev）框架，目标是把“写代码”的能力包装成能够靠代理执行的线性流程，让开发者能专注于需求与评审，而把重复的测试、搜索与补丁交给 Agent。

他指出开放平台让社区可以共建调试工具与示例仓库，从实际 issue 中提取输入、在公开仓库中跟踪 Agent 操作，再把改进反馈给模型。这种闭环让研究能够直接走向工程实践，而不是停留在实验室的基准上。

1.2 赋能普罗大众

他特别提到 2011 年 Mark Andreessen 的文章“Why Software Is Eating the World”，指出各行各业正在被软件重新定义：从娱乐、农业、国防到医疗服务，软件已经逐步取代了物理流程，未来软件负责的领域只会越来越多。这也说明了为什么需要让更多人拥有快速构建软件的能力，而不是让软件开发继续被少数专业工程师垄断。

在介绍动机时，Graham 用近年来多个由软件系统驱动的科研突破作类比，强调当代科学进展正在与代码深度捆绑。对他而言，Agent 的价值在于能把工程能力下放到更广泛的研究者、产品经理与跨学科团队，让“软件创造”成为普遍能力而非稀缺资源。

Mark Andreessen 的预言

2011 年的“Why Software Is Eating the World”认为：软件将成为所有产业服务的新基础设施，而自动化构建软件的能力则是决定竞争力的关键。Graham 整个演讲试图把这句话落地到 Agent 工程中。

Agent 的价值主张

软件开发 Agent 的核心承诺是：将具备代码理解、变更、验证与反馈能力的多模态体系，变成能够像人类开发者一样持续迭代产品的执行者。

1.3 本章小结

- 软件已经主导大部分行业，构建能力的稀缺性推动了 Agent 的实际需求。
- Graham 通过诺贝尔奖与软件领域的对照提醒我们：软件创造已经成为人类科学的核心技能之一。

2 OpenHands 平台与 Agent 架构

2.1 OpenHands 架构组件

OpenHands 提供了一个容器化、可编排的 Agent 开发平台：Agent 能在受控的 sandbox 环境里同时操作终端、编辑器和浏览器，后端可以接入多个 LLM（如 code-davinci 及开源模型），并通过 webhook、测试解释器等工具链完成输入 → 推理 → 行动的闭环。

各组件通过事件总线 and 消息队列通信，允许在运行时替换模型或工具。平台还为每一次动作定义了元数据（触发方式、上下文、预期输出），方便做回滚与调试。

2.2 多模态接口与审计



图 1: OpenHands 画面展示了 Agent 级联的环境感知、LLM 推理与工具执行流。¹

平台强调“可观察性”：每个 Agent 的行动（终端命令、浏览器点击、文件修改）都会被记录，测试结果与日志会流回一个中央审计台。OpenHands 同时提供接口供人工阻断、利用调试视图或插入辅助脚本，这种 plug-and-play 的能力让 Agent 不再是黑盒。

OpenHands 核心能力

OpenHands 包含多模态接入器、LLM 决策引擎、工具执行器（Docker shell/测试/浏览器）以及可视化审计面板。各部分通过事件总线通信，可在运行时替换模型或工具。

可观察性与安全边界

Agent 记录每一个命令，但也必须限制高权限操作，确保沙箱可还原，否则开发日志与环境状态会泄露敏感信息、难以复现。

2.3 本章小结

- OpenHands 把 Agent 的感知、推理、行动与审计串成可插拔的流水线。
- 透明日志与人工干预接口是让工程团队愿意交付 Agent 的关键保障。

¹视频画面时间区间：00:04:45–00:04:50。

3 SWE-bench: 软件工程评测的基准

3.1 Lite 与 Full 的演进

Graham 接下来介绍 SWE-bench: 一个对 Agent 执行 GitHub 实际 Issue 的评测框架。每个样本包含原始仓库、Issue 描述与测试套件, Agent 的任务是理解语义和上下文, 编写补丁并通过该仓库内的测试以验证正确性。

SWE-bench 分为两类: Lite (300 道相对简单的问题) 与 Full (2,294 道来自真实仓库的 Issue)。Lite 集中在理解命令、补丁与基本测试; Full 把问题延展到多文件依赖、跨模块调用与复杂环境。Agent 会被限制在一个上下文窗口内, 因此需要设计有效的提示、索引检索与局部推理策略。

3.2 评估流程与指标

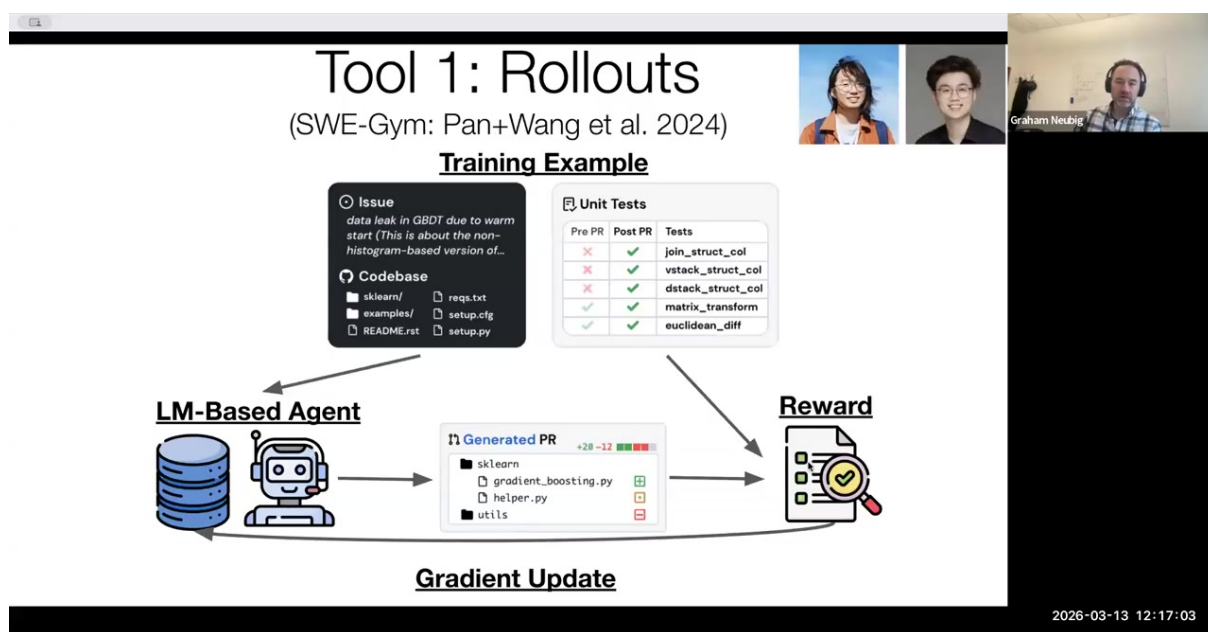


图 2: SWE-bench 的评测统计板块, 展示 Lite 与 Full 数据集的成功率。²

评估流程包含三步: 1) Agent 阅读 Issue 并在仓库中检索相关上下文; 2) 生成并应用补丁 (编辑文件、运行 `git diff`); 3) 运行目标测试, 并把结果与输出日志存档。系统按照 `pass/fail` 统计成功率, 并把失败样本回馈训练流程以改进提示与工具策略。

²视频画面时间区间: 00:15:20-00:15:25。

维度	描述
任务类型	从 Issue 中提取目标、定位相关文件、生成补丁并运行对应测试。
Lite	300 个易错点，旨在验证 prompt 结构与快速修复能力，当前最佳 Agent 通过率在 40%–50% 之间。
Full	2,294 个挑战，包括异步调用、依赖迁移与安全补丁，目前整体通过率低于 20%。

表 1: SWE-bench 的设计维度与当前效果。

SWE-bench 的评估侧重点

SWE-bench 强调工程闭环：Agent 必须提交 patch、运行测试并提供日志，不能只给出 diff；Lite/Full 的划分让研究团队能先优化基础能力再扩展到复杂场景。

量化指标

Graham 强调：Lite 级别的 40% 通过率已经意味着 Agent 能替代部分重复任务，但 Full 级别的低通过率提示我们仍处在“半自动化”时代。

覆盖率局限

SWE-bench 主要评估 coding+ 测试，不包含设计、代码审查、部署和维护等更广的软件工程任务，所评估的能力仍然偏重修复型问题。

3.3 本章小结

- SWE-bench 提供了从简单到复杂的双层跨度，让 Agent 具备逐步演进的目标。
- 当前指标仍然远低于人类工程师，在更广的软件生命周期阶段仍需新增评测。

4 Agent 工程实践：规划、执行与验证

4.1 Plan-Act-Verify 细节

在实际工程中，Agent 必须具备明确的行动循环：先阅读 Issue、调用检索服务锁定上下文、规划补丁、执行代码修改、运行测试、再利用日志反馈调整。Graham 强调多个工具的组合：LLM 侧重生生成，脚本负责操作，测试框架验证结果，最后把证据上传到 dashboard 供人工审核。

他指出，在每一轮 Plan 中应该先生成任务列表，显式地列出哪些文件会被修改、哪些测试要跑。而 Act 阶段则绑定具体的脚本、终端命令，让 Agent 实际写码并执行，Verify 阶段运行测试与静态分析，失败的样本会触发重新 Plan。

4.2 人机协作与工具覆盖

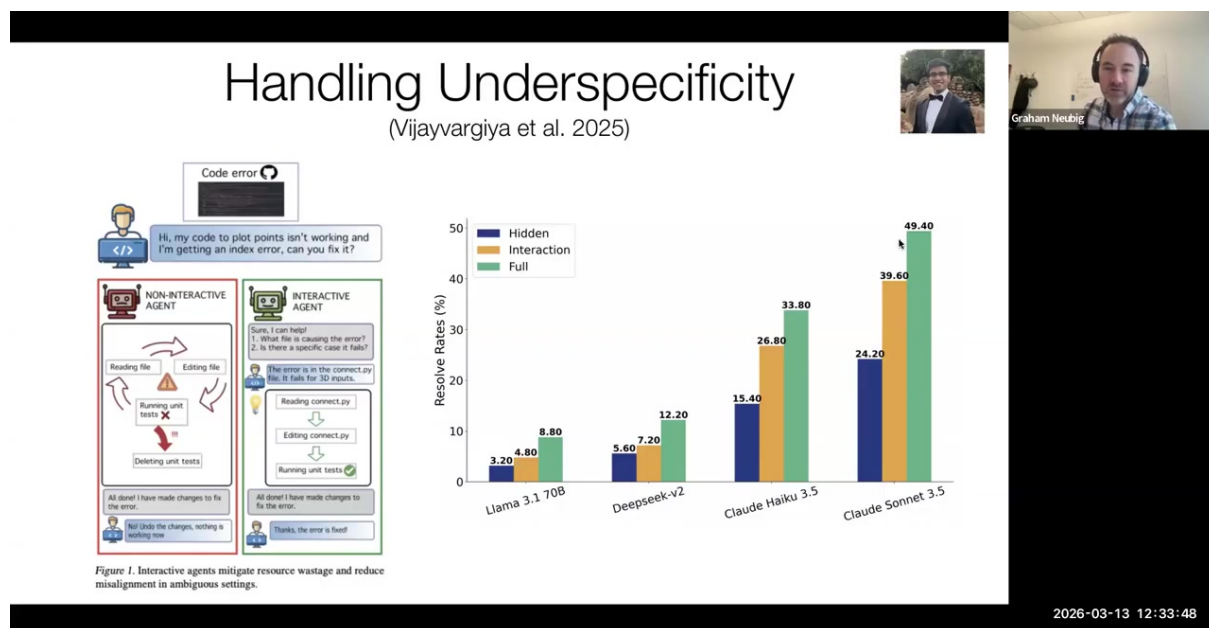


图 3: 人类监督者可以在 Agent 运行期间暂停、检查决策并干预，借此纠正实时错误。³

OpenHands 允许开发者在任何一步插入人工审查：当 Agent 计划执行危险命令（如部署、数据库写入）时，可以暂停流程，查看日志，并在必要时修改提示或手动执行部分命令。长时间运行的任务也会发送 checkpoint，便于追踪状态。

最佳实践

为 Agent 实现 pipeline：先 plan（任务分段、找到依赖），再 act（代码编辑、编译、测试），最后 verify（日志、快照、回滚能力）。失败的步骤需要回溯并把关键上下文传给 LLM。

人工干预的价值

让人类在 Agent 执行时可以观察其行为，并在将要执行危险操作前暂停，是提升可靠性的关键，尤其在发布构建或数据库操作时。

上下文窗口与幻觉

Agent 的上下文有限，如果一次性送入整个仓库结构会超出可读窗口，必须先做信息抽取 / 检索，否则 LLM 会因缺少环境信息产生错觉性的 patch。

4.3 本章小结

- 规划-执行-验证的循环让 Agent 在工程流程中可控。
- 人机协作（暂停、查看日志、跳转上下文）是避免重大误操作的必要手段。

³视频画面时间区间：00:32:05–00:32:10。

5 未来方向与治理挑战

5.1 Agent 训练与数据

Graham 表示，当前的 Agent 训练还没充分利用“Agent 风格”的数据（agent chains、自我反馈、失败纠错），未来需要更多 agent-structured corpora 让模型学会规划与纠错。另一个方向是强化 human-in-the-loop 评估：通过人类与 Agent 的协作走查，让模型学习捕捉并纠正自己的错误。

他补充说，Agent training 应该把过程日志作为训练数据的一部分，让模型不仅看到最终 patch，还能看到 Plan、执行步骤与 Verify 结果。

5.2 治理与安全

未来研究核心

重点在于：训练集成化 agent 行为、引入人类修正轨迹、构建更细粒度的评估（如过程日志是否对齐、补丁的可维护性）。

治理与安全

Agent 过度自动化可能带来高危决策：比如在无人监督下直接部署补丁、跨仓库调用敏感 API，必须引入审批、审计与回滚机制才能量产。

方向拆解

Graham 推荐的三条路径：1) 使用 agent-specific data 训练，2) 构建 / 扩展 SWE-bench 类型的评测，3) 引入实时 human-in-the-loop 反馈与安全机制。

5.3 本章小结

- 解决 agent 训练与评测的空白，是 Agent 进入成熟阶段的突破口。
- 安全、审批与人工干预不应该被视为附加，而是 Agent 生命周期的组成部分。

6 工程案例：Agent 参与的真实仓库

6.1 半自动化提交

Graham 分享了自己的仓库实践：有些项目中“一半的提交内容都是先由编码 Agent 草拟，再由他去检查与修复”的。Agent 先读 Issue，通过摘要与检索找到相关文件，然后草稿出初步补丁，最后由人类工程师合并。

他预计未来一两年内，Agent 会承担更多日常琐碎任务，让工程师把注意力放在高层设计。现在 Agent 还能顺利处理共享依赖的 Package 脚本、基础 CLI 操作与简单的 bug fix，复杂场景仍然需要人工复审。

6.2 过程回顾与反馈

每次 Agent 运行后，团队都会记录最终日志：包括主动 fetch 的文件、调用的命令、测试结果与失败原因。这些日志被转成训练样本与回调提醒，让 Agent 学会把失败样本中有用的步骤嵌入提

示链。

他还强调：“观察 Agent 运行的过程并给出即时纠正，比在结果上打分更能提升模型质量。”这种观察式反馈包括人工暂停、注入提示、在纠错后再让 Agent 重试。

案例经验

Graham 已经在多个仓库里使用 Agent 提笔，用 Agent 贡献的提交占据约一半工作量。这证明 Agent 在重复性任务中已具备实用价值。

实用建议

先把 Agent 投放在 10 分钟以内的任务（如格式化、补丁、依赖升级），拼凑人类 review 的流程，再逐渐向 longer-running job 扩展。

不要让 Agent 独自部署

即便 Agent 可以完成某些改动，也不应让其单独推到生产，必须保留人工审查、测试回归与手动 approve。

6.3 本章小结

- Agent 在真实仓库中的尝试已展现半自动化的潜力，但还不足以完全替代人类工程师。
- 记录过程日志与即时纠错是提升 Agent 可靠性的关键做法。

7 可观察性与调试实践

7.1 日志与监控流水线

为了让 Agent 的每次运行变得可追踪，OpenHands 会把终端、浏览器、测试、日志都抽象成统一的事件流。每条事件都包含时间戳、调用的工具、上下文快照与预期结果，便于工程师查找出错点。

- 终端日志：记录每一个 shell 命令及其返回码，并会与测试结果一同上传。
- 浏览器与网络交互：捕捉点击、表单提交与请求响应，便于复现 HTTP 会话与 API 流程。
- 文件/代码 diff：自动保存 Agent 修改的 diff 片段、相关文件路径和上下文。

层级	用途	示例
命令级	捕捉每个 shell 操作与结果	<code>pip install</code> 、 <code>pytest tests/foo.py</code>
上下文级	记录使用的上下文版本与检索数据	啮合的 Git commit + Issue summary
结果级	归档测试通过/失败、Diff 分析	Unit test failed with stack trace

表 2: OpenHands 观测层级与日志用途。

7.2 调试模式与快速回滚

当 Agent 计划执行高风险命令（如发布、数据库迁移）时，系统会自动切入调试模式：暂停流程、捕捉当前快照，并把命令提醒人类确认。若命令执行失败，平台可以基于日志和 diff 自动回滚，避免污染主分支。

调试建议

让 Agent 把每一步的“意图”和“预期结果”写入日志，可以加速人工排查并形成训练样本。

滥用日志可能泄露敏感信息

调试日志中可能包含 API Key、path、stack，必须设置脱敏与访问权限，避免在审计时泄露隐私。

7.3 本章小结

- 通过统一的事件流与模板化日志，可以快速缩小故障范围。
- 调试模式提供了人工确认与自动回滚的双重保障。

8 代理指标与质量门控

8.1 关键指标

除了 SWE-bench 的成功率之外，还需要关注其他指标：人类干预率、平均等待时间、测试覆盖与差异率。这些指标帮助团队判断 Agent 是否真的减轻了工程负担而非增加额外开销。

指标拆解

Graham 建议关注四个维度：1) 成功率（pass/fail），2) 人类干预率（多少次需要暂停），3) 运行时间（是否在可接受范围），4) 可解释性（是否输出合理解释）。

8.2 质量门控节点

在 Agent pipeline 中，质量门控可以设在 Plan、Act 或 Verify 节点：Plan 需要人工 check 生成的步骤，Act 阶段需审查要执行的命令，Verify 阶段则是测试与静态分析。只有所有门控通过后，才允许推送 patch。

门控节点	判据	响应
Plan	计划的步骤是否合法、有无权限操作	人类审查或追加提示
Act	命令是否涉及敏感资源（数据库、云服务）	暂停并请求审批
Verify	测试/静态分析是否全部通过	回滚、增加测试或人工复审

表 3: 质量门控在 Agent 流程中的典型布置。

不要只看成功率

高成功率可能意味着 Agent 被限制在低风险任务，实际生产环境还需要综合人类评估与运行成本。

8.3 本章小结

- 成功率只是其中一个指标，需要结合干预率、运行时间等综合评估。
- 质量门控帮助在关键节点介入，防止 Agent 直接发布并引入风险。

9 术语表与工具清单

9.1 常见术语

为了方便团队协作，Graham 推荐维护术语表：如 Plan-Act-Verify、Agent Chain、Observability Stream 等，统一语言可以减少误解。

- Plan-Act-Verify：明确划分 Agent 的计划、执行与验证步骤。
- Agent Chain：把多个 Agent 组合成复杂任务的链式执行流程。
- Observability Stream：Agent 输出的日志/命令/测试结果的统一数据流。

9.2 工具清单

OpenHands 内置了多套工具：Terminal Runner、Browser Controller、Test Executor、Diff Reporter。部署时可以为每个保持单独的环境与权限，并记录使用情况。

工具	用途
Terminal Runner	运行 shell 命令并记录 stdout/stderr。
Browser Controller	进行页面交互、抓取数据并回写环境。
Test Executor	运行 pytest、npm test 等并收集结果。
Diff Reporter	汇总 patch、生成 markdown 描述与差异。

表 4: OpenHands 主要工具及其角色。

工具组合策略

尽量在不同的步骤中使用不同权限的工具，降低误操作风险；比如把数据库操作限制在单独的命令中，并设置回滚逻辑。

9.3 本章小结

- 术语表让跨团队合作更顺畅，避免对 Agent 意图的误解。
- 工具清单帮助把不同类型的操作模块化，便于设置权限与审计。

10 工程协作与变更流程

10.1 项目管理视角

与 Agent 协同工作的团队需要重新定义“任务状态”：Agent 生成的 patch 需要被纳入待审列表，并与 backlog、issue、任务分解保持一致。Graham 建议用“人类负责需求 + Agent 负责执行”的方式把需求拆成多个小任务，然后把 Agent 的活动写进 sprint board。

- 将 Agent 的计划阶段视为“开发任务”，由 PM 或 tech lead 审查。
- 在 Agent Act 阶段，保留人工确认机制，确保仅执行经过权限批准的命令。
- Verify 阶段把测试报告贴到 issue，以供 QA 回归审查。

10.2 交付与风险沟通

当 Agent 帮助实现 patch 时，交付状态应明确标识“Agent 草案 + 人工确认”。如果 Agent 失败，则需要自动通知相关 owner，附带日志与 diff，以便及时接手。

沟通矩阵

快速记录：Agent 失败 → 通知 owner；Agent 成功但需要 review → 跟进 PR；Agent 执行危险命令 → 请求审批。

不要把 Agent 当成黑盒

在沟通中要说明 Agent 正在做什么、为什么做，从而减少团队对其信任的恐惧，并确保在失败时能快速接替。

10.3 本章小结

- 项目管理需要把 Agent 输出作为第一类工作项并纳入看板。
- 风险沟通与审批机制是 Agent 大规模部署的前提。

11 访谈内容速览

11.1 0:00–15:00：背景与平台

开场部分强调了 All Hands AI 与 OpenHands 的背景，强调将 Agent 的能力下放给更广泛的团队。Graham 讲述了一个 GitHub “绿色”哲学：希望看到越来越多的 commits 由 Agent 初稿生成，再通过人类优化。

- 0–15 分钟：背景与平台定位。
- 15–30 分钟：SWE-bench 的设计与实验数据。
- 30–45 分钟：实践案例、人工干预与调试。
- 45–60 分钟：未来研究与治理的路线图。

11.2 本章小结

- 访谈按时间轴展开，涵盖动机、基准、实践、未来。
- 每部分都强调 “Agent 需要人类参与” 这一贯穿主线。

12 部署与监控实践

12.1 CI/CD 集成

Graham 建议把 Agent 与 SWE-bench 结合到 CI 流水线：每天或每个 commit 运行 Lite 集合，并把失败样本生成 issue 供模型团队研究。自动化脚本可以在多个环境中并行执行 Lite/Full，并上传日志供人类审查。

```
1 #!/usr/bin/env bash
2 set -ex
3 cd /workspace/swe-bench
4 python run_swe_bench.py --split lite --model codegen
5 python run_swe_bench.py --split full --tests smoke
6 tar czf swe-bench-results.tar.gz outputs/
7 gzip -c outputs/logs/latest.log > swe-bench-results/logs/latest.log.gz
8 notify-team --subject "SWE-bench_run_finished" --files swe-bench-results.tar.gz
```

脚本中的 `notify-team` 可以接入 Slack/Email，把失败 diff 以及通用输入输出一起发送。

CI/CD 运行策略

把 Agent 运行拆成 “预热、执行、回放”：预热阶段重建 repo 与依赖，执行阶段运行 Agent，回放阶段把日志 + diff 上传供开发者复审。

12.2 小规模实验

在将 Agent 扩大部署前，先在 `sandbox` 仓库做 1-2 个试点任务，用 Agent 生成 patch，但只在内网 review，并保持每轮运行在 15 分钟以内，确保可控。

短期实验建议

保留人工检查点：只允许 Agent 修改少于 3 个文件，所有操作必须在 `git` 里打 patch 后由人类确认。

避免主分支污染

不要让 Agent 直接 push 到 ‘main’/‘master’；应该在 feature 分支生成 patch，待 review 完成后再合并。

12.3 本章小结

- 把 Agent 嵌入 CI 后能定期捕捉 regressions，并把失败 trace 回 feed。
- 小规模的内网实验可以降低迭代风险并积累经验。

13 访谈建议与常见问题

13.1 观众提问回顾

Graham 回答了几个经常出现的问题：Agent 能否处理非代码任务？如何衡量 Agent 成功率？什么时候需要人工接管？他的回答体现出对“人机协作”核心信念：Agent 负责重复、可编排的部分，人类保留审查与风险控制的权限。

- “Agent 能否做需求分析？” → 能辅助整理文本，但仍需 PM 追问盲点。
- “失败时如何接管？” → Agent 生成的 diff 会被 human in the loop 复审，不通过即回滚。
- “如何保留 traceability？” → 所有动作都由 OpenHands 记录，日志里有时间戳、命令、结果。

13.2 建议清单

主题	建议
需求对齐	把需求拆成细小任务，并在 Plan 阶段写明预期输出。
日志使能	让 Agent 输出“意图 + 结果”到日志，便于人工分析。
安全门控	把数据库操作和部署命令设置为需要人工 approve 的步骤。

表 5: 常见建议的速览。

13.3 本章小结

- 访谈 Q&A 再次强调：Agent 不是替代，而是协助人类工程师的伙伴。
- 明确的建议清单便于团队把访谈内容落地。

14 数据洞察与评估仪表盘

14.1 核心仪表盘视图

Graham 建议构建一套 Agent 评估仪表盘，用以实时监控成功率、干预次数、平均运行时长与安全门控触发记录。仪表板的每一项指标都应该关联具体的事件 log，以便在指标异常时追溯根因。

指标	说明	目标
通过率	Agent 执行 patch 并通过测试的比例	≥ 40% (Lite)、≥ 20% (Full)
人工干预率	需要人类暂停或审批的比例	≤ 10%
运行时长	平均从 Plan 到 Verify 完成的时间	≤ 20 分钟
安全触发	涉及数据库/部署的命令数量	需人工审核

表 6: Agent 仪表板中关注的核心指标。

仪表盘数据依赖

每个指标都应 与事件 log（命令、diff、测试输出）关联，便于回溯并训练可解释的模型。

14.2 仪表盘反馈循环

仪表盘不仅监控，还应支持“反馈 -> 调整”流程：当某个指标出现回落，系统自动把失败 log 发送给 Agent 团队，让他们出具新的 prompt/工具组合，并把新策略推送到测试环境。

数据反馈实践

设计自动化报告：指标异常（如通过率 <30%）时，自动打开 issue、附上 diff/log 并提醒团队，防止问题长期沉没。

14.3 本章小结

- 实时仪表盘是判断 Agent 成熟度的关键窗户。
- 把异常事件变成自动反馈循环，可以加速策略迭代。

15 团队文化与培训

15.1 文化落地策略

Graham 强调：Agent 没有强力表现的时候，人们会怀疑其价值；因此必须在团队内部传播“Agent + 人类”的文化，举例说明 Agent 负责重复任务，而人类保留最终审查。每个 sprint 复盘时都要展示 Agent 成果。

1. 每周分享 Agent 成功案例（如 Agent 生成 diff 并通过 3 次测试）。
2. 制定“Agent Etiquette”文档，明确什么时候可以调用 Agent，哪里必须人工复盘。
3. 设立 Mentor 制度，让新成员在熟悉 Agent 工具前由经验工程师带领。

15.2 培训与指南

培训材料应包含基本术语（Plan-Act-Verify/Observability Stream）、常用 scripts 与风险提示。培训结束后可以安排实战演练：让学员在 sandbox 中让 Agent 处理一个 Issue，并在旁边观察、记录干预点。

培训常见挑战

新成员容易过度依赖 Agent，建议在实战练习中设置“人工复审”门槛，确保他们熟悉如何验证 Agent 的输出。

知识传承

把每次人机协作的流程记录成文档与视频，便于跨团队共享经验，减少“Agent 只能某个人才能用”的风险。

15.3 本章小结

- 文化、培训与实战演练共同支撑 Agent 的长期落地。
- 明确的 etiquette 与复盘机制可以消除对 Agent 的不信任。

16 安全治理与审查流程

16.1 审计链设计

OpenHands 生成的每一次命令、文件 diff、测试结果都被送入审计链，由审计服务写入事件中间件。这让团队可以在事后轻松查看“谁让 Agent 执行了哪条命令”，并与安全策略对照。

审计阶段	内容	输出
事件捕捉	命令、浏览、文件操作	Log ID + 时间戳
策略对齐	检查是否触发敏感资源	Warning / Approval request
报告归档	生成邮件/issue	可复现的审核记录

表 7: 安全审计流程中的三个阶段。

审计要素

记录要包含上下文（Issue、diff、测试），便于人类快速判断风险。

16.2 审批与权限

当 Agent 计划执行敏感命令（比如数据库迁移、云部署）时，审批模块会自动触发。审批决策包含触发命令、调用环境、预期结果、相关 diff 以及上一次的测试状态，审批者只需确认或拒绝。

审批的节奏

不要过度审批以致阻塞迭代；而在高风险场景必须设置 multi-party approval，防止单点失效。

16.3 本章小结

- OpenHands 的审计链把 Agent 行为录下来，用于安全复盘。
- 合理设置审批与权限可以在不降低效率的情况下强化安全。

17 实战回顾与案例模板

17.1 代码修复示例

Graham 描述了一个典型案例：Agent 发现 bug 之后，Plan 阶段写下要修改的文件列表，Act 阶段使用 `python apply_patch` 生成 diff，Verify 阶段跑 `pytest`。全流程记录在日志里，若测试失败就自动回滚。

示例流程

Plan: 提取 Issue, 列出文件与函数; Act: 运行 Agent 写 patch, 保存 diff; Verify: 跑测试, 若失败把 log 推送到 issue。

17.2 失败与纠错记录

团队会把每次失败样本记录为“失败矩阵”: 包含失败原因(测试、权限、语法)、Agent 输出、人工干预内容。该矩阵是训练 prompt 的重要素材。

失败日志不要删

失败日志包含关键语义信息, 用于生成 agent chains 训练数据; 删除会导致未来无法复现故障。

17.3 本章小结

- 通过模版化流程可以把单次 success 定义为“Plan + Act + Verify”三段完成。
- 记录失败矩阵为后续训练与 prompt 调整提供依据。

18 事件响应与复盘

18.1 故障归因流程

每次 Agent 失败后团队会立即启动复盘: 首先提取‘failure matrix’包含命令、触发文件、执行环境及返回码; 然后比对当前策略与历史策略, 判断是 prompt、工具链还是运行环境的回归。

分类	典型触发条件	复盘动作
Prompt 失效	生成与上下文不符	把 prompt 重新模板化, 并加入新的 examples
工具链错误	命令/测试挂	检查 docker 镜像、依赖版本、权限
评测环境差异	本地通过但 CI 失败	对比环境变量和数据版本, 做 dev vs prod tests

表 8: 故障分类与复盘动作。

18.2 复盘纪要传播

复盘结果会以 Markdown 形式写入团队 wiki, 内容包括时间、负责人、复盘结论和后续动作, 并通过 Slack 频道 #agent-postmortem 发送提醒。每次复盘还会带上关键截图(如测试 log、diff)让未来快速定位。

不要让复盘沉没

复盘纪要必须在 24 小时内发布，过久会导致场景遗忘。

复盘价值

复盘不仅是找错，更是把 Agent 的失败样本变成训练 prompt 的课件，把“为什么失败”转化成可重复的规律。

18.3 本章小结

- 快速归因与复盘可防止同类问题反复出现。
- 把复盘结果分享给团队可提升整体 Agent 工程成熟度。

19 实施检查与持续改进

19.1 常规检查表

Graham 建议在每次 Agent 迭代前后执行检查表，确保 prompt、工具链、日志和审批机制都处于可用状态。检查表项包括：是否更新 open-source requirements、是否验证镜像权限、是否更新测试基线、是否同步 Slack 通知。

检查项	描述
Prompt 状态	确保 prompt 模板包含最新的 issue 样本与 guardrails
工具链更新	检查 docker 镜像、依赖版本与 API 权限是否过期
日志与审计	验证事件流是否正常，并且 audit log 可回放
审批机制	复核高风险命令审批流程是否完整，包括 emergency override

表 9: 实施前后的常规检查项。

19.2 持续改进循环

每个 Agent 迭代都应该包含一个回顾环节：回顾成功的案例、失败的经验，并把新的 insight 写入 FAQ/文档，供下一轮模型训练使用。Graham 强调 “1

小步快跑

把 Agent 更新分解成 5 分钟的改动，验证通过后再合并，尽量避免一次性大量变更。

改进指标

使用“成功案例比例”、“复盘完成率”和“文档更新次数”作为改进指标，把工程和研究的工作闭环在一起。

19.3 本章小结

- 检查表保障 Agent 架构在每次迭代前后都处于健康状态。
- 小步快跑与改进指标让团队可以在风险可控的前提下持续进步。

20 总结与延伸

Graham 的这场访谈可以归结为三个平行推进的议题：架构、评测与治理。每条道路都需要大量工程实践、全面评估与安全反思，才能让 Agent 在高风险的软件开发中被广泛采用。

主题	要点
背景与动机	软件正在吞噬各行业，把“写代码”的能力下放给 Agent 能让更多团队带着工程思维去解决问题。
架构与评测	OpenHands 提供透明 Agent 平台，SWE-bench Lite/Full 提供逐步递进的工程评估指标。
未来与治理	需要 agent 风格的数据、更多评测维度与内建的安全机制，才能让 Agent 安全进入主流开发流程。

表 10: 本次演讲的三个核心要素。

进一步阅读：

- OpenHands 开源仓库：<https://github.com/OpenHands/OpenHands>。
- SWE-bench 论文：Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” ICLR 2024。
- All Hands AI 官方博客与社区反馈入口，关注 agent 安全与人机协作议题。

20.1 本章小结

- 演讲强调的架构、评测与治理三条线必须并行推进，才能把 Agent 安全推向生产。
- 关注 OpenHands 与 SWE-bench 的演化，有助于把代理式软件开发落地。